WEB TECHNOLOGIES

Unit 1: Protocols, Web Servers, and Web Design Principles

Rgpvnotes.tech - Your Academic Excellence Partner

UNIT 1: Protocols and Programs | Secure Connections | Application & Development Tools | The Web Browser | What is a Server | Setting up UNIX/LINUX Servers | Logging Users & Dynamic IP | Web Site Design Principles & Navigation

Table of Contents

Topic 1: Protocols and Programs

Topic 2: Secure Connections (SSL/TLS & HTTPS)

Topic 3: Application and Development Tools

Topic 4: The Web Browser

Topic 5: What is a Server? Setting up UNIX/LINUX Web Servers

Topic 6: Logging Users and Dynamic IP

Topic 7: Web Design Principles, Planning & Navigation

Revision Notes

Important Questions

1 Protocols and Programs

Definition:

A **Protocol** is a set of formal rules and conventions that govern how data is formatted, transmitted, and received over a network. A **Program** (in the context of networks) is the software application that implements these protocols to provide network services to users.

Detailed Explanation:

In web technologies, communication between a client (like your computer) and a server relies heavily on

network protocols. Without protocols, devices wouldn't understand the electronic signals they send to one another. Protocols dictate the speed of transmission, the structure of the data packets, error detection mechanisms, and how connections are initiated and terminated. The Internet operates primarily on the TCP/IP suite (Transmission Control Protocol/Internet Protocol).

Several high-level application protocols run on top of TCP/IP to perform specific tasks. The most prominent is **HTTP (Hypertext Transfer Protocol)**, which dictates how web pages (HTML documents) are requested by browsers and delivered by web servers. **FTP (File Transfer Protocol)** is used for transferring files between a client and a server. **SMTP (Simple Mail Transfer Protocol)** is utilized for sending emails.

Programs are the software clients and servers that "speak" these protocols. For instance, a web browser (like Chrome) is a program that speaks HTTP. A FileZilla client is a program that speaks FTP. On the server side, Apache or Nginx are programs that listen for HTTP requests and serve the appropriate files based on the protocol's rules.

🔑 Key Protocols in Web Technologies:

- ✓ **HTTP/HTTPS:** Hypertext Transfer Protocol (Secure). Used for accessing websites and web services.
- ✓ **TCP/IP:** The foundational communication language of the Internet. TCP ensures reliable delivery, IP handles routing.
- ✓ **FTP:** File Transfer Protocol. Used to upload website files to a web server.
- ✓ **DNS:** Domain Name System. Translates human-readable domain names (google.com) into IP addresses (142.250.190.46).
- ✓ **SMTP/POP3/IMAP:** Protocols dedicated to sending and retrieving electronic mail.

📊 Protocol Stack Interaction

```
[Application Layer] -> HTTP, FTP, SMTP, DNS (Web Browsers, Email Clients)
                                        |
   [Transport Layer]   -> TCP, UDP (Reliable/Unreliable data transfer)
                                        |
       [Internet Layer]    -> IP (Routing packets across networks)
                                        |
 [Network Access]    -> Ethernet, Wi-Fi (Physical transmission of bits)
```

💡 Real-Life Example:

Think of **Protocols** as the grammar and rules of a human language (e.g., English syntax). Think of **Programs** as two people talking to each other. If Person A (the Web Browser) speaks English (HTTP) and Person B (the Web Server) also speaks English (HTTP), they can successfully exchange a document (the Web Page).

Hinglish Explanation:

*Internet pe devices aapas mein kaise baat karenge, uske rules ko **Protocol** kehte hain. Jaise website kholne ke liye HTTP use hota hai. Aur jo software in rules ko follow karke kaam karta hai (jaise Chrome browser ya Apache server), use **Program** kehte hain. Bina protocols ke, data sirf kachra (garbage) ban jayega kyunki computer use samajh nahi payega.*

2 Secure Connections (HTTPS, SSL/TLS)

Definition:

Secure Connections refer to encrypted communication channels over a computer network, preventing eavesdropping and tampering. In web technologies, this is primarily achieved using **HTTPS**, which is HTTP layered over **SSL/TLS** (Secure Sockets Layer / Transport Layer Security) encryption protocols.

Detailed Explanation:

When you browse the web using standard HTTP, data is sent in "plaintext." If a hacker intercepts the connection (a Man-in-the-Middle attack), they can easily read passwords, credit card numbers, and personal messages. Secure connections solve this by encrypting the data before it leaves your computer and decrypting it only when it reaches the intended server.

SSL (Secure Sockets Layer) and its modern successor **TLS (Transport Layer Security)** are cryptographic protocols that provide this security. When a browser connects to a secure server (HTTPS), they perform an "SSL/TLS Handshake." During this handshake, the server proves its identity by presenting a **Digital Certificate** (issued by a trusted Certificate Authority or CA). Once verified, the client and server agree on a unique "Session Key" to encrypt all subsequent data.

Secure connections use a mix of *Asymmetric Cryptography* (Public/Private keys) for the initial handshake to safely share the session key, and *Symmetric Cryptography* (the Session Key) for the actual data transfer because it is much faster for large amounts of data.

Key Elements of Secure Connections:

- ✓ **Encryption:** Scrambling data so only authorized parties can read it.
- ✓ **Data Integrity:** Ensuring that data has not been altered or corrupted in transit.
- ✓ **Authentication:** Proving that the server you are talking to is the legitimate owner of the website.
- ✓ **Digital Certificate:** An electronic document used to prove the ownership of a public key (includes site info, owner, CA signature).
- ✓ **Port 443:** The default network port used for HTTPS traffic (standard HTTP uses Port 80).

📊 The SSL/TLS Handshake Process

```
Client (Browser)                          Server
        |                                    |
        | ---- 1. "Client Hello" (Ciphers) --> |
        |                                    |
        | <--- 2. "Server Hello" + Cert ------ |
        |                                    |
        | ---- 3. Verifies Cert, Sends     --> |
        |         Encrypted Session Key      |
        |                                    |
        | <--- 4. "Finished" (Secure!) ------- |
        |                                    |
       [======= ENCRYPTED DATA TRANSFER =======]
```

✅ Advantages:

✓ Protects sensitive user data (passwords, banking details).

✓ Builds user trust (padlock icon in browser).

✓ Improves SEO (Google ranks HTTPS sites higher).

❌ Disadvantages:

✗ Requires purchasing/managing Digital Certificates (though Let's Encrypt offers free ones).

✗ Slight computational overhead for encryption/decryption.

👉 ***Hinglish Explanation:***
Normal HTTP pe data plain text mein jata hai, jise koi bhi hacker raste mein padh sakta hai. ***HTTPS*** *aur* ***SSL/TLS*** *us data ko encrypt (lock) kar dete hain. Server ek "Certificate" dikhata hai apni pehchan prove karne ke liye. Jab padlock (tala) ka icon dikhta hai, iska matlab aapka data hacker se safe hai.*

3 Application and Development Tools

📌 Definition:

Application and Development Tools are software programs, utilities, and environments that web developers use to write, test, debug, and deploy web applications efficiently. These tools streamline the software development lifecycle (SDLC).

📖 Detailed Explanation:

Building modern web applications requires more than just a simple text editor like Notepad. Developers rely on a complex ecosystem of tools to write clean code, manage dependencies, test APIs, and collaborate with teams. The choice of development tools drastically affects a developer's productivity and the final product's quality.

At the core of development are **IDEs (Integrated Development Environments)** and Code Editors. Tools like Visual Studio Code (VS Code) or WebStorm provide syntax highlighting, auto-completion, and integrated terminal access. To manage code versions and collaborate, developers use **Version Control Systems (VCS)** like Git, hosted on platforms like GitHub or GitLab.

Furthermore, web development involves client-server architecture. Developers use tools like **Postman** or Insomnia to test backend APIs without writing frontend code. For frontend debugging, every modern browser comes with **Developer Tools (DevTools)** (e.g., Chrome DevTools) allowing developers to inspect HTML/CSS, monitor network requests, and debug JavaScript in real-time. Finally, build tools and package managers (like npm, Webpack) automate repetitive tasks like minifying code and managing external libraries.

🔧 Essential Categories of Development Tools:

- ✓ **Code Editors / IDEs:** VS Code, Sublime Text, IntelliJ IDEA, Eclipse. Used for writing code with assistance.
- ✓ **Browser DevTools:** Built into Chrome/Firefox. Used to inspect DOM, test CSS, and debug JavaScript errors.
- ✓ **Version Control Systems (VCS):** Git. Used to track changes, revert mistakes, and collaborate.
- ✓ **API Testing Tools:** Postman, Swagger. Used to send HTTP requests to test backend endpoints.
- ✓ **Package Managers:** npm (Node Package Manager), Yarn. Used to download and manage external code libraries.
- ✓ **Local Servers environments:** XAMPP, WAMP, Docker. Used to run backend code and databases locally before deploying to the internet.

🌍 Real-World Applications:

→ A frontend developer uses **Chrome DevTools** to fix a broken CSS layout on a website.

→ A team uses **Git and GitHub** to merge their code together without overwriting each other's work.

→ A backend developer uses **Postman** to ensure the "User Login" API returns the correct security token before building the frontend interface.

👉 Hinglish Explanation:

*Web development sirf code likhna nahi hai; uske liye special tools chahiye hote hain. **VS Code** jaise editors code likhne mein help karte hain. **Chrome DevTools** se hum website ki errors (bugs) dhoondte hain. **Git** code ka backup rakhta hai, aur **Postman** se hum server APIs ko test karte hain bina frontend banaye. Ye sab tools developer ki life aasan banate hain.*

4 The Web Browser

Definition:

A **Web Browser** is an application software used on the client side to access, retrieve, and display information from the World Wide Web. It translates raw HTML, CSS, and JavaScript delivered by web servers into interactive visual web pages.

Detailed Explanation:

The web browser is the gateway to the Internet for most users. When a user types a URL (Uniform Resource Locator) into the address bar, the browser acts as a client. It first resolves the domain name to an IP address using DNS, then initiates an HTTP/HTTPS request to the target web server. Once the server responds with data, the browser's primary job begins: Rendering.

Inside the browser, several components work together. The **Browser Engine** bridges the UI (User Interface) and the rendering engine. The **Rendering Engine** (like Blink in Chrome, Gecko in Firefox, WebKit in Safari) parses the HTML to create the DOM (Document Object Model) tree and parses CSS to create the CSSOM (CSS Object Model). It then combines these to paint the pixels on the screen. Simultaneously, the **JavaScript Engine** (like V8 in Chrome) executes any dynamic scripts, which can modify the DOM and change the page dynamically without reloading. Browsers also contain networking layers for HTTP requests, UI backends for drawing basic widgets, and data persistence layers (like LocalStorage, Cookies, and IndexedDB) to save data locally on the user's machine.

Main Components of a Web Browser:

- ✓ **User Interface (UI):** Address bar, back/forward buttons, bookmarks.
- ✓ **Browser Engine:** Coordinates actions between the UI and the rendering engine.
- ✓ **Rendering Engine:** Parses HTML/CSS and displays the requested content on the screen.
- ✓ **Networking:** Handles network calls like HTTP requests, managing security and caching.
- ✓ **JavaScript Interpreter/Engine:** Parses and executes JavaScript code.
- ✓ **Data Storage:** Manages local data like Cookies, LocalStorage, and browser cache.

High-Level Browser Architecture

```
+-----------------------------------------------------+
|                    User Interface                   |
+-----------------------------------------------------+
|                    Browser Engine                   |
+-----------------------------------------------------+
```

```
|                    Rendering Engine                    |
+-------------+-------------+-------------+----------+
|  Networking | JS Engine   | UI Backend  | Data     |
|             |             |             | Storage  |
+-------------+-------------+-------------+----------+
```

How it Works (Real-Life Example):

When you type "facebook.com": 1. Browser asks DNS for Facebook's IP. 2. Browser sends an HTTP GET request to that IP. 3. Server sends back HTML code. 4. **Rendering Engine** reads the HTML, sees a link to a CSS file, requests it, and paints the blue header. 5. **JS Engine** runs scripts to load your notifications without refreshing the page.

Hinglish Explanation:
*Web Browser (jaise Chrome, Firefox) ek software hai jo internet ki files ko aapki screen par dikhata hai. Jab aap website ka naam dalte hain, browser server se HTML/CSS/JS file mangwata hai. Fir uska **Rendering Engine** us code ko padh kar design banata hai, aur **JavaScript Engine** website ko interactive (jaise button click pe popup aana) banata hai.*

5 What is a Server? Setting up UNIX and LINUX Web Servers

Definition:

A **Server** is a computer program or a device that provides functionality, resources, or data to other programs or devices, known as "clients." A **Web Server** specifically stores, processes, and delivers web pages to users over the internet using HTTP/HTTPS.

Detailed Explanation:

The term "server" can refer to physical hardware (a powerful computer sitting in a data center) or the software running on that hardware. In the context of web technologies, a Web Server is software (like Apache HTTP Server, Nginx, or Microsoft IIS) that listens on specific network ports (Port 80 for HTTP, 443 for HTTPS). When a request arrives from a client browser, the web server locates the requested file (like index.html), reads it, and transmits it back to the client.
UNIX and LINUX operating systems are the industry standard for hosting web servers due to their stability, security, open-source nature, and low resource consumption. Setting up a Linux web server typically involves configuring the **LAMP Stack** (Linux, Apache, MySQL, PHP/Python).
Setting up involves accessing the Linux machine (usually via SSH), using a package manager (like `apt` for Ubuntu/Debian or `yum` for CentOS/RHEL) to install the web server software, opening the necessary firewall ports, and configuring the server blocks (Virtual Hosts) to tell the server which folder holds the website

files for a specific domain name.

🔧 Steps to Set Up a Basic Apache Server on Linux (Ubuntu):

✓ **Step 1: Update Packages.** `sudo apt update` (Ensures you have the latest software lists).

✓ **Step 2: Install Apache.** `sudo apt install apache2` (Downloads and installs the web server).

✓ **Step 3: Adjust Firewall.** `sudo ufw allow 'Apache'` (Opens Port 80 to allow internet traffic).

✓ **Step 4: Check Status.** `sudo systemctl status apache2` (Verifies the server is running).

✓ **Step 5: Add Content.** Place your HTML files in the default directory: `/var/www/html/`.

✓ **Step 6: Virtual Hosts.** Configure `.conf` files in `/etc/apache2/sites-available/` to host multiple domains on one server.

✅ Advantages of Linux Web Servers:

✓ **Cost-Effective:** Most Linux distributions and web servers (Apache/Nginx) are free and open-source.

✓ **Security & Stability:** Linux handles high traffic loads without crashing and has robust file permissions.

✓ **Flexibility:** Command-line interface allows heavy automation and scripting.

❌ Disadvantages:

✗ **Steep Learning Curve:** Requires knowledge of command-line interface (CLI) and shell commands.

✗ No native Graphical User Interface (GUI) on most server distributions.

👉 ***Hinglish Explanation:***
Server *ek aisi machine ya software hai jo 24/7 internet se connect rehti hai aur clients (browsers) ki requests ka reply karti hai. Zyadatar servers* ***Linux/Unix*** *OS pe chalte hain kyunki wo free, fast aur secure hote hain. Linux par website host karne ke liye hume Apache ya Nginx jaisa software install karna padta hai, firewall open karni padti hai, aur apni HTML files ko* `/var/www/html` *folder mein rakhna padta hai.*

6 Logging Users and Dynamic IP

📌 Definition:

Logging Users is the process of recording server activity, including visitor requests, errors, and system events, into log files for analysis and security. A **Dynamic IP** is a temporary IP address assigned to a device by a network (via DHCP), which changes over time, as opposed to a fixed Static IP.

📖 Detailed Explanation (Logging Users):

Every time a user visits a website, the web server generates a record of that interaction. This is crucial for web administrators to monitor traffic, debug issues, and track malicious activities. In servers like Apache or Nginx, there are two primary log files: the **Access Log** (records who visited, when, what page they requested, their IP address, and their browser type) and the **Error Log** (records server malfunctions, missing files, or failed scripts).

Log analysis tools (like AWStats, GoAccess, or modern platforms like Google Analytics) parse these raw text files to create visual reports. By analyzing the logged IP addresses, User-Agent strings, and session IDs (cookies), administrators can track unique user journeys across the website.

📖 Detailed Explanation (Dynamic IP & Servers):

For a server to be reliably found on the internet, it ideally needs a Static (permanent) IP address. However, Internet Service Providers (ISPs) often assign **Dynamic IPs** to regular broadband connections using DHCP (Dynamic Host Configuration Protocol). This means the IP address changes every time the router restarts or after a lease expires.

If you want to host a web server on a machine with a Dynamic IP, your domain name will stop working whenever the IP changes. The solution is **Dynamic DNS (DDNS)**. DDNS is a service where your server runs a small program that constantly checks its current IP. If the IP changes, the program instantly updates the DNS records with the DDNS provider, ensuring that users typing your domain name are always routed to your new, updated IP address.

🔑 Key Concepts in Logging & Dynamic IP:

- ✓ **access.log:** Records successful and failed HTTP requests (Status codes like 200 OK, 404 Not Found).
- ✓ **error.log:** Diagnostic information for server crashes or misconfigurations (Status 500 Internal Server Error).
- ✓ **Log Formats:** Common Log Format (CLF) or Combined Log Format, which includes referrer and User-Agent data.
- ✓ **DHCP:** The protocol responsible for automatically assigning Dynamic IPs to devices on a network.
- ✓ **DDNS (Dynamic DNS):** A system that maps a static domain name to a constantly changing Dynamic IP.

💡 Example of an Access Log Entry:

```
192.168.1.10 - - [10/Apr/2026:14:32:01 +0000] "GET /index.html HTTP/1.1" 200 1043 "http://google.com" "Mozilla/5.0 (Windows NT 10.0)"
```

Translation: User with IP 192.168.1.10 requested the home page on April 10 at 2:32 PM. The request was successful (200), transferred 1043 bytes. The user came from Google using a Windows 10 computer.

👉 *Hinglish Explanation:*

Logging: *Web server ek diary maintain karta hai jisme likha hota hai ki website pe kaun aaya, kis time aaya, aur kya open kiya (Access log), aur kya errors aayi (Error log). Isse admin traffic aur hackers ko track karta hai.*

Dynamic IP: *Aam internet connection ka IP address change hota rehta hai (Dynamic IP). Agar aap aise connection pe server chalate hain, toh URL kaam karna band kar dega jab IP change hoga. Is problem ko solve karne ke liye* ***DDNS (Dynamic DNS)*** *use karte hain, jo automatic naya IP domain ke sath update kar deta hai.*

7 Web Site Design Principles, Planning, and Navigation

Definition:

Web Site Design Principles are guidelines that ensure a website is aesthetically pleasing, highly usable, and accessible. **Planning and Navigation** involve organizing the website's structure, defining the target audience, and creating logical pathways (menus/links) so users can easily find information.

Web Site Design Principles:

Good web design goes beyond making a site look "pretty"; it focuses on User Experience (UX) and User Interface (UI). Designers follow established principles to create intuitive layouts. A popular framework is the **CRAP principle**: Contrast (making elements stand out), Repetition (consistent branding and styles), Alignment (creating visual order), and Proximity (grouping related items together). Furthermore, modern design mandates **Responsive Design**, ensuring the website adapts seamlessly to different screen sizes (mobile, tablet, desktop).

Planning the Site:

A successful website requires meticulous planning before writing any code. This phase includes defining the **Purpose** (e.g., e-commerce, blog, corporate site) and analyzing the **Target Audience** to tailor the design and content appropriately. Designers create *Wireframes* (low-fidelity sketches of page layouts) and *Mockups* (high-fidelity visual designs). They also define technical requirements, budget, and hosting needs.

Site Navigation:

Navigation is how users move from one page to another. If users can't find what they need within 3 clicks (The 3-Click Rule), they often leave the site. Planning navigation starts with creating a **Sitemap**, a hierarchical diagram of all website pages. Navigation menus must be placed in predictable locations (top header or left sidebar), use clear descriptive labels (e.g., "Contact Us" instead of "Interact"), and indicate the user's current location (breadcrumbs: Home > Electronics > Laptops).

Key Navigation Structures:

- ✓ **Hierarchical (Tree):** The most common structure. Starts at a Home page and branches out into categories and subcategories.
- ✓ **Linear (Sequential):** Users move through pages in a specific order (Page 1 -> Page 2 -> Page 3). Good for tutorials, checkout processes, or wizards.

- ✓ **Web (Network):** Pages are heavily linked to each other without a strict order. Good for wikis (like Wikipedia).

🔍 Crucial Design Principles:

- ✓ **Simplicity & Clarity:** Avoid clutter; use whitespace effectively to let content breathe.
- ✓ **Visual Hierarchy:** Use size, color, and fonts to draw the eye to the most important elements first (like Call-to-Action buttons).
- ✓ **Consistency:** Keep headers, footers, and color schemes uniform across all pages.
- ✓ **Accessibility:** Design for all users, including those with disabilities (high contrast, readable fonts, alt-text for images).

📊 Hierarchical Navigation Sitemap

```
                    [ HOME PAGE ]
                          |
     +--------------------+--------------------+
     |                    |                    |
[ ABOUT US ]        [ PRODUCTS ]          [ CONTACT ]
                 |                    |
            [ Team ]          +-------+-------+
                              |               |
                       [ Laptops ]     [ Smartphones ]
```

🌍 Application in Real-World:

→ **Amazon** uses a deep *Hierarchical Navigation* system (mega-menus) and *Breadcrumbs* to help users navigate millions of products without getting lost.

→ A payment gateway uses a *Linear Navigation* (Cart -> Address -> Payment -> Success) to ensure the user completes the transaction in the correct order.

👉 ***Hinglish Explanation:***
Website design sirf sundar dikhna nahi hota, use aasan (user-friendly) bhi banana padta hai. ***Planning*** *mein hum decide karte hain ki website kis audience ke liye hai aur uski zarurat kya hai.* ***Navigation*** *ka matlab hai website ke menus aur links; ye itne clear hone chahiye ki user ko jo chahiye wo max 3 clicks mein mil jaye. Agar menus confusing honge, to user website chhod dega.*

🔁 QUICK REVISION NOTES

1. Protocols & Programs:
Protocols (HTTP, FTP, TCP/IP) are rules for data transfer. Programs (Browsers, Web Servers) are software that implement these rules to communicate.

2. Secure Connections:
HTTPS uses SSL/TLS certificates to encrypt data between client and server, preventing hackers from reading sensitive information during transmission.

3. App & Dev Tools:
Tools used to build/debug software. Include IDEs (VS Code), Version Control (Git), Browser DevTools (for frontend debugging), and API testers (Postman).

4. The Web Browser:
Client software that fetches files via HTTP. Uses a Rendering Engine to parse HTML/CSS and paint pixels, and a JS Engine for dynamic interactivity.

5. Linux Web Servers:
Software (Apache/Nginx) running on Linux hardware. Handles HTTP requests, reads files, and serves them to clients. Known for high security, cost-effectiveness, and stability.

6. Logging & Dynamic IP:
Access logs record visitor details (IP, time, URL); Error logs record crashes. Dynamic IPs change frequently (DHCP); DDNS maps a changing IP to a fixed domain name so servers remain accessible.

7. Web Design & Navigation:
Design principles ensure usability (Simplicity, Consistency, Visual Hierarchy). Planning involves defining audience and wireframes. Navigation structures (Hierarchical, Linear) guide users logically through the site.

IMPORTANT QUESTIONS (6 Marks Each)

Q1. Differentiate between a Protocol and a Program in the context of web technologies. List three protocols and explain their primary use. 6 Marks

Expected Answer Points:

- Define Protocol: Rules for formatting and transmitting data over networks.
- Define Program: Software application implementing protocols (e.g., Browser, Apache).
- Protocol 1: HTTP/HTTPS - Used for requesting and serving web pages securely.
- Protocol 2: FTP - Used for uploading/downloading files to a server.
- Protocol 3: DNS - Resolves domain names into IP addresses.

Q2. How do secure connections work on the web? Explain the role of SSL/TLS and HTTPS. 6 Marks

Expected Answer Points:

- Definition of HTTPS and how it differs from HTTP (encryption).
- Explain SSL/TLS protocol purpose: Data Encryption, Integrity, and Authentication.
- Describe the SSL Handshake briefly (Client hello, server certificate, session key exchange).
- Role of Digital Certificates and Certificate Authorities (CAs).

Q3. Explain the architecture of a Web Browser. What are its major components and how do they function to display a webpage? 6 Marks

Expected Answer Points:

- Define a Web Browser as a client-side program.
- List major components: User Interface, Browser Engine, Rendering Engine, JS Engine, Networking, Data Storage.
- Explain Rendering Engine: Parses HTML into DOM, CSS into CSSOM, and paints the screen.
- Explain JS Engine: Executes dynamic scripts to modify the DOM.

Q4. What is a Web Server? Describe the general steps to set up a basic Apache web server on a UNIX/LINUX environment. 6 Marks

Expected Answer Points:

- Define Web Server: Hardware/software that listens on port 80/443 and serves web files via HTTP.
- Why Linux: Stability, security, open-source (LAMP stack).
- Step 1: Install via package manager (`apt install apache2`).
- Step 2: Configure firewall to allow HTTP traffic.
- Step 3: Start/Enable the service and place website files in root directory (`/var/www/html`).

Q5. Discuss the principles of Website Design. Explain different types of navigation structures used while planning a website. 6 Marks

Expected Answer Points:

- Design Principles: Simplicity, Consistency, Visual Hierarchy, CRAP (Contrast, Repetition, Alignment, Proximity), Accessibility.
- Planning phase: Target audience analysis, purpose definition, wireframing/sitemaps.
- Hierarchical Navigation: Tree-like structure (Home -> Categories -> Subcategories). Most common.

- Linear Navigation: Step-by-step sequential flow (e.g., checkout processes).
- Importance of navigation: The 3-click rule, breadcrumbs, clear labels.

📚 Unit 1 notes are comprehensive and exam-ready 📚

Master the fundamentals of Web Technologies and score high marks!

Web Technologies - Unit 1 Notes

📚 Source: Rgpvnotes.tech | Premium Academic Content for Students

© 2026 - Your Academic Excellence Partner

These notes are created for educational purposes to help students understand complex concepts in a structured manner.